Fundamentos de Programación

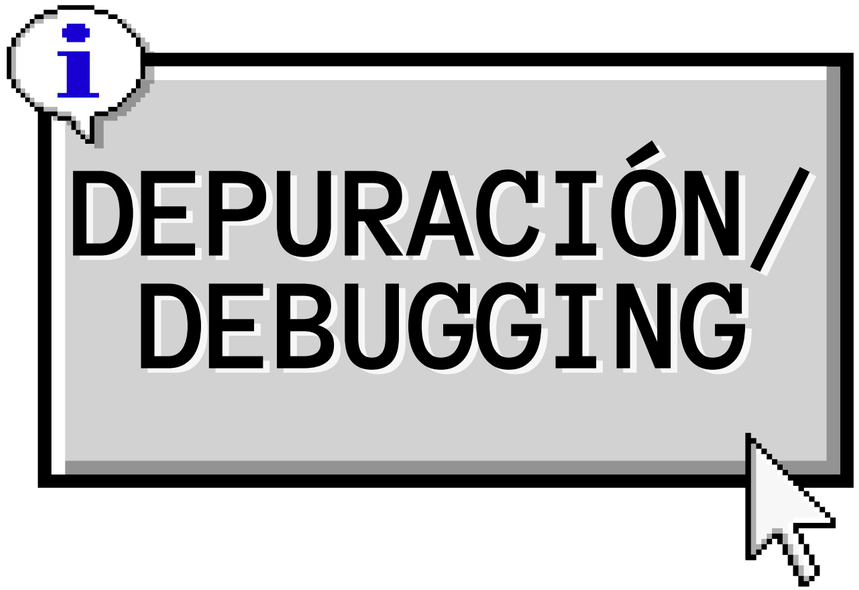
Home Content Contact

# DEBUGGING

DAVID VILLAMIL BORJAS



Start




Es el proceso de revisar, identificar y corregir errores en el código fuente de un programa para asegurarse de que funcione exactamente como se diseñó.

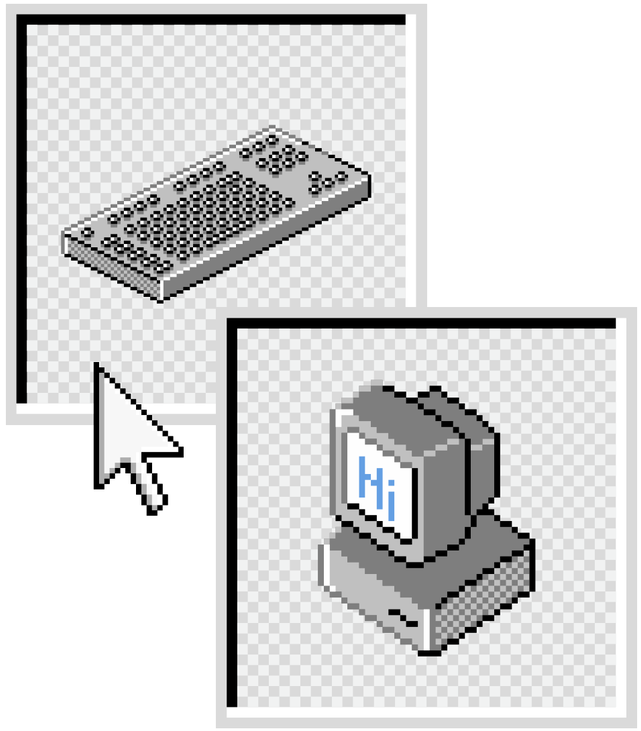
# ORIGEN

Proviene de la ingeniera Grace Hopper, quien encontró una polilla (bug) atascada en un componente de una computadora analógica que causaba un fallo físico. Desde entonces, los errores informáticos se conocen como bugs y su solución como debugging.



# IMPORTANCIA

Corregir errores garantiza la calidad y fiabilidad del software. Un error no corregido puede acumularse y causar desde datos incorrectos en un sistema, hasta pérdidas financieras graves o fallos en sistemas críticos.



Fundamentos de programación

Home **Content** Contact




# TIPOS DE ERRORES

# SINTÁCTICOS

Las palabras no forman una sentencia válida para Python; por ende, esta clase de inconvenientes se depuran gracias a los editores disponibles en la herramienta, en este caso, IDLE.

```
#Programa que dice “Hola Mundo”
Print(“Hola Mundo”)
```

```
Traceback (most recent call last):
File "<pyshell#1>", line 2, in <module>
Print("Hola Mundo")
NameError: name 'Print' is not defined. Did you mean: 'print'?
```



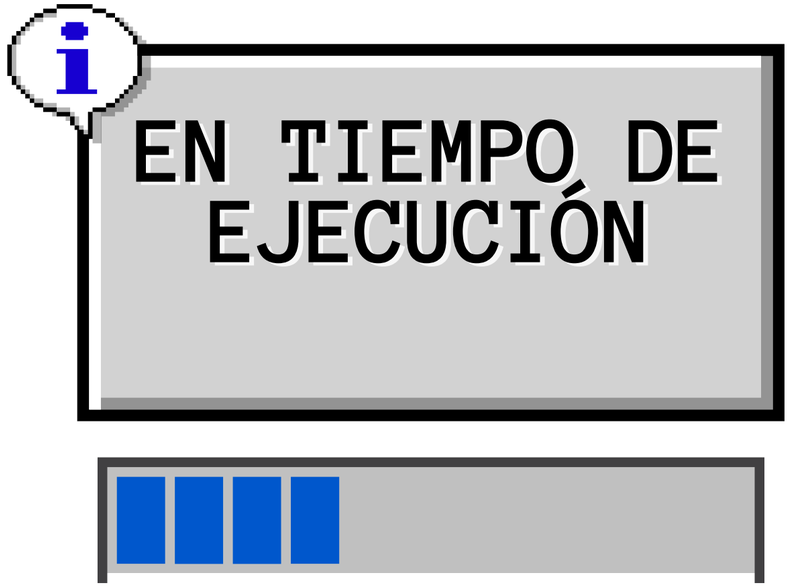
# SEMÁNTICOS/LÓGICA

Los errores semánticos (también llamados errores de lógica) ocurren cuando la estructura de tu código está perfectamente escrita según las reglas del lenguaje (no hay faltas de ortografía ni fallas que detengan el programa de golpe), pero el resultado obtenido no es el que querías. A diferencia de los errores sintácticos o de ejecución, la computadora no sabe que hay un problema porque ella solo sigue tus instrucciones al pie de la letra.

```
promedio = a + b + c / 3
```

```
if edad > 18:
print("Eres mayor de edad")
```

En pleno funcionamiento, la computadora intenta realizar una instrucción imposible o inválida y se detiene de golpe (se rompe o "crashea").




```
edad = int(input("Ingresa tu edad: "))
```

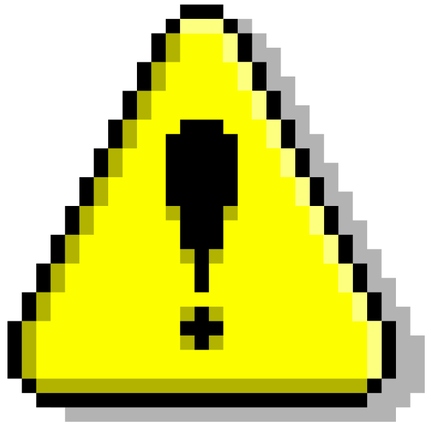
```
a = float(input("Primer número: "))
b = float(input("Segundo número: "))
print("La división es:", a / b)
```

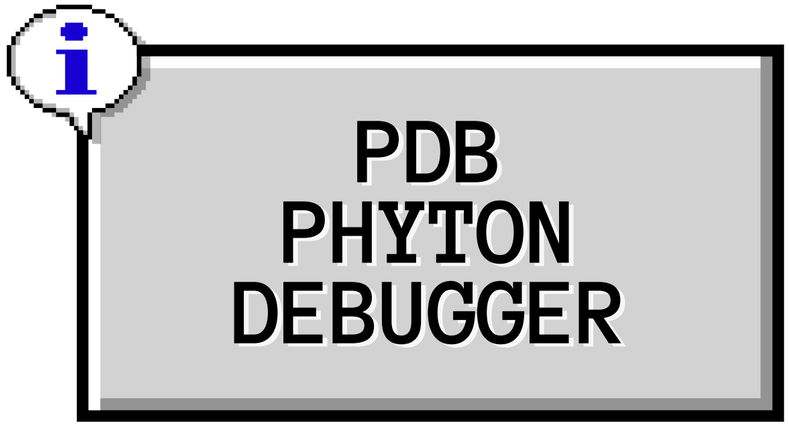
Fundamentos de programación

Home **Content** Contact




# PROCESO DE DEPURACIÓN






```
import pdb; pdb.set_trace()
```

- `list` (o `l`): Muestra el mapa de código alrededor de la línea donde el programa está pausado. Verás una flecha ( `->` ) indicando la instrucción exacta que se va a ejecutar a continuación.
- `next` (o `n`): Avanza **una línea** de código. Ejecuta la instrucción actual y se detiene inmediatamente en la siguiente.
- `p <variable>`: Significa *print*. Sirve para **imprimir el valor actual** de una o varias variables (ej. `p total` o `p total, i`). Si intentas consultar una variable antes de que se cree, PDB te dirá que no está definida.
- `break <número_de_línea>` (o `b`): Pone un **punto de interrupción (breakpoint)** en una línea específica para que no tengas que ir dando saltos individuales. Por ejemplo, `break 20` le dice a Python que corra normalmente pero se vuelva a pausar al llegar a la línea 20.
- `continue` (o `c`): Le da **"Play"** a la ejecución. El programa corre a velocidad normal hasta encontrarse con el siguiente *breakpoint* o hasta finalizar por completo.

# EJEMPLO DE DEPURACIÓN

```
import pdb; pdb.set_trace()

total = 0
numero = int(input("Dame el número: ")) #
Ejemplo: 5

for i in range(numero):
    total += i

print(total)
```



Flujo de Depuración Paso a Paso (Explicación con PDB)

Paso 1: Iniciar y listar el código

Comando: list

Muestra las líneas del programa e indica con una flecha (->) la instrucción actual.

Paso 2: Avanzar e inspeccionar

- Comando: next
- Comando: p total  Muestra 0 (confirma que la variable fue inicializada).

Paso 3: Punto de interrupción (Breakpoint)

- Comando: break 20 (o la línea del for) y luego continue.
- Solicita el número al usuario y detiene la ejecución justo antes de sumar.

Paso 4: Detección del error

- Comando: p total, i $\rightarrow$ Muestra (0, 0).
- Falla encontrada: En Python, range(5) genera la secuencia 0, 1, 2, 3, 4. No llega al 5, por lo que la suma final da 10 en lugar de 15.

# SOLUCIÓN AL ERROR LÓGICO

Para incluir el número límite ingresado por el usuario, el ciclo debe llegar hasta numero + 1:

```
# Código corregido
for i in range(1, numero + 1):
    total += i
```

Timmerman Industries

Home Content **Contact**

# GRACIAS

